

Force-Directed Graph Layout

Ema Čikotić
UP FAMNIT
Koper, Slovenia
89221136@student.upr.si

Abstract

Force-directed graph drawing algorithms provide a way to visualize complex graphs by positioning vertices according to simulated physical forces. However, their computational cost can become significant as the size of the graph increases. This project implements the Fruchterman–Reingold force-directed graph layout algorithm in Java using three computational approaches: sequential, parallel, and distributed. The parallel implementation uses multiple threads to accelerate force calculations, while the distributed implementation uses MPJ Express to divide the computation among multiple processes. The implementations are evaluated on randomly generated graphs by varying the number of vertices and edges and, for the distributed version, the number of processes. Performance is evaluated using execution time, speedup, and parallel efficiency. The results show that parallelization provides greater performance benefits as the number of vertices increases, while the distributed implementation demonstrates non-linear scaling due to the overhead associated with communication and synchronization between processes.

1 Introduction

Graphs are widely used to represent relationships between entities in areas such as social networks, communication networks, and other interconnected systems. As graphs become larger and more complex, effective visualization becomes important for understanding their structure. Force-directed graph drawing algorithms address this problem by arranging vertices according to simulated physical forces to produce visually meaningful layouts.

The Fruchterman–Reingold algorithm uses repulsive forces between vertices and attractive forces between connected vertices to iteratively determine their positions. A significant part of its computational cost comes from calculating repulsive forces between pairs of vertices. As the number of vertices increases, these calculations become increasingly expensive, making the algorithm suitable for exploring parallel and distributed computation.

This project implements the Fruchterman–Reingold algorithm in Java using sequential, parallel, and distributed approaches. The project supports random graph generation and visualization, configurable graph and layout parameters, and performance measurement. The implementations are evaluated by varying the number of vertices and edges, while the distributed implementation is additionally evaluated using different numbers of processes. The aim is to compare the performance and scalability of the three computational approaches.

2 Fruchterman–Reingold Algorithm

The Fruchterman–Reingold algorithm models graph drawing as a physical system in which vertices interact through attractive and

repulsive forces. Repulsive forces act between all pairs of vertices, while attractive forces act only between vertices connected by an edge.

For two vertices separated by a distance x , the repulsive force is defined as

$$F_R(x) = \frac{K^2}{x},$$

while the attractive force between connected vertices is

$$F_A(x) = \frac{x^2}{K}.$$

The parameter K represents the preferred distance between vertices and is calculated as

$$K = C \sqrt{\frac{\text{area}}{|V|}},$$

where C is a configurable constant, area is the drawing area, and $|V|$ is the number of vertices.

During each iteration, the forces acting on the vertices are calculated and accumulated as displacement vectors, which are then used to update their positions. A temperature parameter limits the maximum movement of a vertex and is gradually reduced, allowing larger movements at the beginning and smaller adjustments as the layout stabilizes.

The repulsive-force calculation considers pairs of vertices and has a computational complexity of $O(|V|^2)$ per iteration, while attractive forces are calculated for the existing edges in $O(|E|)$. The overall computational complexity per iteration is therefore

$$O(|V|^2 + |E|).$$

The quadratic repulsive-force calculation becomes increasingly expensive as the number of vertices grows and therefore represents the main opportunity for parallelization.

3 Implementation

The project implements three versions of the Fruchterman–Reingold algorithm in Java: sequential, parallel, and distributed. All three implementations use the same graph representation and underlying layout algorithm, allowing their performance to be compared under equivalent conditions.

3.1 Graph Generation and Visualization

Graphs are generated randomly according to a user-specified number of vertices and edges. Each vertex is assigned an initial random position within the dimensions of the drawing area, while edges are created between randomly selected vertices. A configurable random seed makes graph generation reproducible, allowing the same graph to be used when comparing different implementations.

A graphical interface allows the user to configure the width and height of the drawing area, the constant C , the number of vertices, the number of edges, and the random seed. The generated graph is then displayed in the visualization panel.

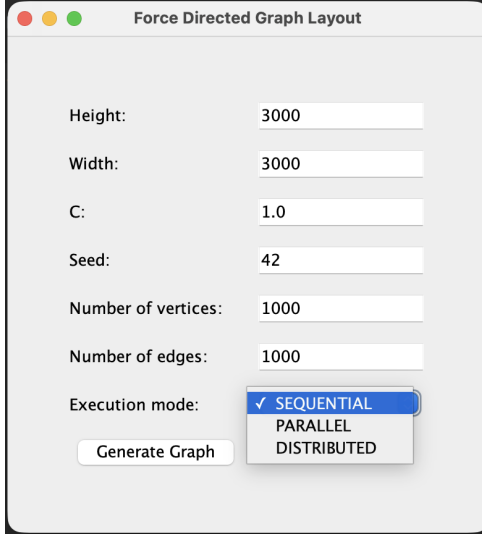


Figure 1: Graph generation interface with configurable layout parameters and execution mode.

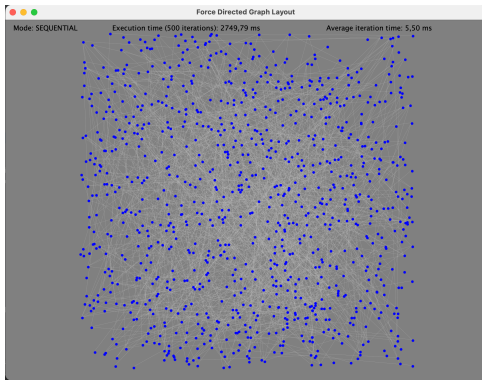


Figure 2: Initial randomly generated graph with 1000 vertices and 1000 edges.

The layout is updated iteratively, allowing the graph to gradually transform from its initial random arrangement into the final force-directed layout. The visualization is updated during this process so that the movement of the vertices can be observed as the algorithm progresses.

3.2 Sequential Implementation

The sequential implementation serves as the baseline for performance comparison. During each iteration, all force calculations and vertex updates are performed sequentially by a single thread.

First, the displacement of each vertex is initialized to zero. Repulsive forces are calculated between pairs of vertices and accumulated

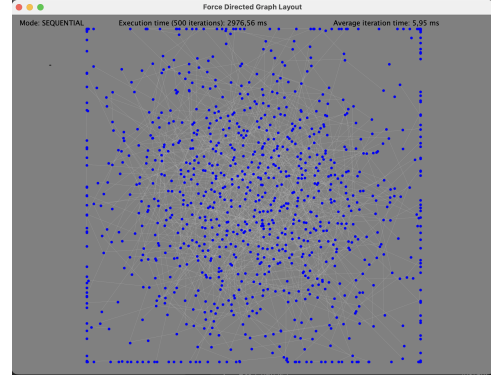


Figure 3: Graph with 1000 vertices and 1000 edges after applying the Fruchterman-Reingold algorithm.

as displacement vectors. Attractive forces are then calculated for each edge and added to the corresponding vertex displacements.

Once the forces have been calculated, each vertex position is updated according to its resulting displacement. Movement is limited by the current temperature, and vertex positions are constrained to remain within the boundaries of the drawing area. The temperature is gradually reduced as the algorithm progresses. These steps are repeated for each iteration of the algorithm.

3.3 Parallel Implementation

The parallel implementation was developed to reduce the execution time of the computationally intensive parts of the Fruchterman-Reingold algorithm. A fixed thread pool is created based on the number of available processors, using one fewer worker thread than the number of detected processors.

The vertices are divided into chunks among the worker threads for the calculation of repulsive forces. Each worker processes its assigned vertices independently, calculating their interactions with the remaining vertices. Attractive-force calculations are also performed in parallel by dividing the edges into chunks. Each worker produces local displacement results, which are combined after the tasks have completed.

Vertex position updates are similarly divided among the worker threads. Synchronization between these computational stages ensures that all required calculations are completed before the algorithm proceeds to the next stage and eventually to the next iteration.

The parallel implementation follows the same force model and cooling process as the sequential implementation while distributing computational work across multiple processor cores. The effectiveness of parallelization depends on the size of the workload, since thread management, synchronization, and the combination of intermediate results introduce additional overhead.

3.4 Distributed Implementation

The distributed implementation divides the computation among multiple independent processes using MPJ Express, a Java implementation of the Message Passing Interface (MPI) model. Each process is assigned a unique rank, and the vertices are divided into chunks among the available processes.

During each iteration, each process calculates the repulsive forces for its assigned subset of vertices. Since these calculations depend on the positions of all other vertices, each process maintains access to the current positions of the complete graph. Attractive-force contributions are also calculated locally for the vertices assigned to each process.

After the force calculations are completed, the displacement values calculated by the individual processes are gathered by the root process. The root process combines these results and updates the positions of all vertices. The updated positions are then broadcast to all processes so that each process has the same graph state before the next iteration begins.

This communication and synchronization are repeated during every iteration. Unlike the shared-memory parallel implementation, the distributed version therefore introduces additional communication overhead between processes. Its performance was evaluated using different numbers of processes to examine how this overhead affects scalability, speedup, and efficiency.

4 Experimental Methodology

The sequential, parallel, and distributed implementations were evaluated using a series of benchmark experiments. All experiments used the same Fruchterman–Reingold algorithm parameters and reproducible randomly generated graphs to allow meaningful performance comparisons between the implementations.

Two sets of experiments were performed to examine the effect of graph size. In the first set, the number of vertices was varied between 500, 1000, 2000, and 3000 while the number of edges was fixed at 1000. In the second set, the number of edges was varied between 500, 1000, 2000, and 3000 while the number of vertices was fixed at 1000. This makes it possible to examine separately how the number of vertices and edges affects execution time.

For the performance experiments, each benchmark run executed the layout algorithm for 500 iterations. Before collecting measurements, 50 warm-up iterations were performed for each implementation to reduce the influence of JVM initialization and Just-In-Time compilation. Each configuration was measured five times, and the average execution time across the five runs was used for the performance analysis.

The experiments were conducted on a MacBook Air with an Apple M2 processor (8 cores: 4 performance and 4 efficiency cores) and 8 GB of memory. The implementations were executed using OpenJDK 21.0.8. The distributed implementation used MPJ Express 0.44.

For the sequential and parallel implementations, total execution time and average execution time per iteration were measured. Speedup was calculated as

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}},$$

where T_{seq} is the sequential execution time and T_{par} is the parallel execution time.

The distributed implementation was evaluated using one, two, three, and four MPJ processes. Distributed speedup was calculated relative to the execution time of the one-process distributed implementation:

$$S_p = \frac{T_1}{T_p},$$

where T_1 is the execution time using one process and T_p is the execution time using p processes. Parallel efficiency was calculated as

$$E_p = \frac{S_p}{p}.$$

These measurements were used to evaluate the performance and scalability of the parallel and distributed implementations.

5 Results and Performance Analysis

5.1 Sequential and Parallel Performance

The parallel implementation consistently reduced execution time compared with the sequential implementation, with the performance improvement becoming more significant as the number of vertices increased. For 500 vertices and 1000 edges, execution time decreased from 928.188 ms to 359.619 ms, corresponding to a speedup of 2.581. At 3000 vertices, execution time decreased from 50269.394 ms to 8365.612 ms, resulting in a speedup of 6.009.

The increasing speedup indicates that larger workloads benefit more from parallel execution. This is consistent with the $O(|V|^2)$ complexity of the repulsive-force calculation, as increasing the number of vertices substantially increases the amount of pairwise computation that can be performed in parallel.

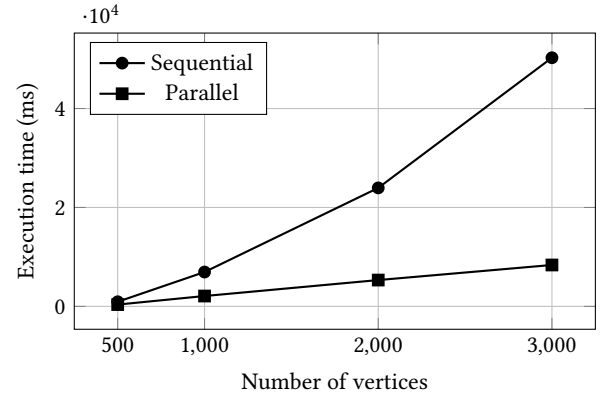


Figure 4: Sequential and parallel execution time with an increasing number of vertices and 1000 fixed edges.

5.1.1 Effect of the Number of Edges. When the number of vertices was fixed at 1000, increasing the number of edges had a much smaller effect on execution time. The sequential execution time remained between approximately 2557 ms and 2642 ms, while the parallel execution time remained between approximately 578 ms and 615 ms across the tested edge counts.

This is consistent with the algorithm’s computational complexity. Since repulsive-force calculations have a complexity of $O(|V|^2)$ and attractive-force calculations have a complexity of $O(|E|)$, keeping the number of vertices fixed while increasing only the number of edges produced a comparatively small change in the overall computational workload.

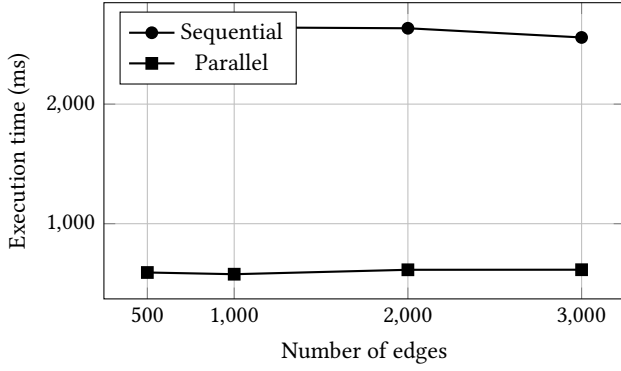


Figure 5: Sequential and parallel execution time with an increasing number of edges and 1000 fixed vertices.

5.1.2 Parallel Speedup. The speedup achieved by the parallel implementation increased with the number of vertices, from 2.581 for 500 vertices to 3.322 for 1000 vertices, 4.504 for 2000 vertices, and 6.009 for 3000 vertices. This trend shows that the benefits of parallel execution increase as the computational workload grows.

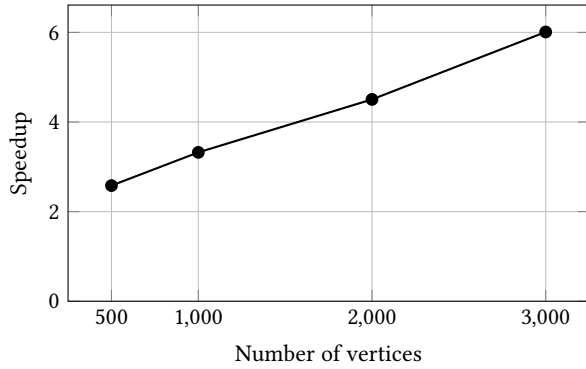


Figure 6: Speedup of the parallel implementation relative to the sequential implementation for increasing numbers of vertices.

5.2 Distributed Performance and Scalability

The distributed implementation was evaluated using one to four MPI processes. In general, distributing the computation across multiple processes reduced execution time, although the improvement was not proportional to the number of processes.

For the largest tested vertex configuration, with 3000 vertices and 1000 edges, execution time decreased from 30673.176 ms with one process to 23168.602 ms with two processes, 11163.237 ms with three processes, and 9915.939 ms with four processes. This corresponds to a maximum distributed speedup of 3.093 for this configuration.

Increasing the number of processes did not improve performance in every case. For example, with 1000 vertices and 1000 edges, three processes completed the computation in 1152.808 ms, while four processes required 1336.100 ms. This suggests that the additional

communication and synchronization overhead can outweigh the benefit of using an additional process for some workloads.

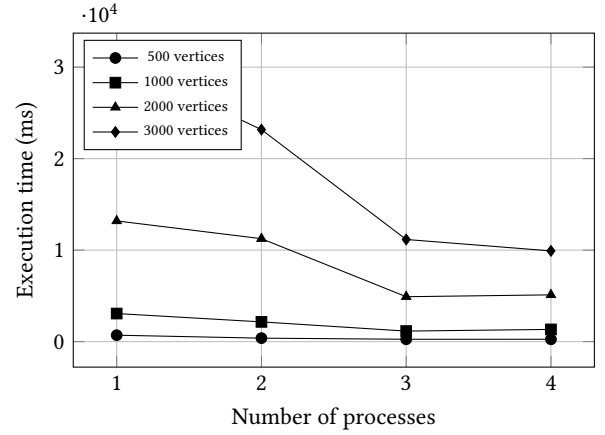


Figure 7: Distributed execution time using one to four processes for different vertex counts with 1000 fixed edges.

5.2.1 Distributed Speedup and Efficiency. The distributed speedup generally increased when three or four processes were used, although the corresponding efficiency shows that additional processes were not always utilized equally effectively.

With three processes, the vertex experiments achieved speedups between 2.663 and 2.748 for graphs containing 1000 to 3000 vertices, corresponding to efficiencies of approximately 88.76% to 91.59%. In contrast, the four-process configurations generally achieved lower efficiency. For example, the 1000-vertex experiment achieved a speedup of 2.297 with four processes, corresponding to an efficiency of 57.44%.

For the largest tested vertex configuration, four processes achieved the highest absolute performance, with a speedup of 3.093 and an efficiency of 77.33%. These results show that using more processes does not necessarily result in higher efficiency, as performance depends on the balance between the computational workload and the overhead introduced by communication and synchronizations.

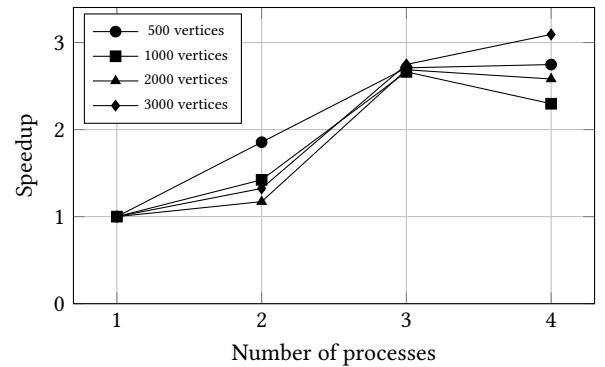


Figure 8: Distributed speedup for different vertex counts using one to four processes.

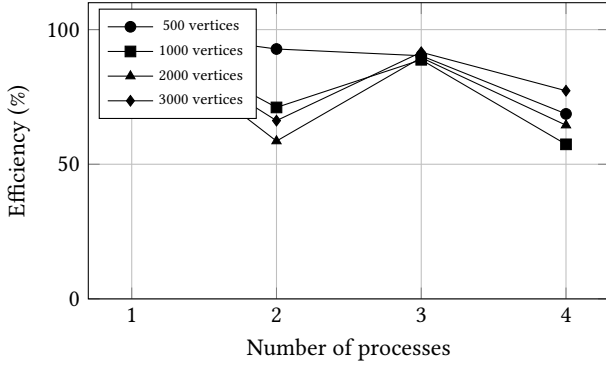


Figure 9: Distributed parallel efficiency for different vertex counts using one to four processes.

5.3 Comparison of Sequential, Parallel, and Distributed Implementations

A final comparison was performed between the sequential, parallel, and distributed implementations. For the distributed implementation, the results obtained using four processes were used. Figure 10 compares the execution times as the number of vertices increases while the number of edges remains fixed at 1000.

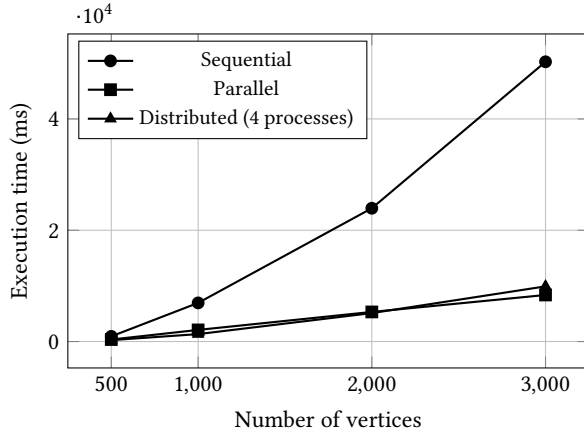


Figure 10: Execution time comparison of the sequential, parallel, and distributed implementations with 1000 fixed edges.

Both the parallel and distributed implementations substantially reduced execution time compared with the sequential implementation. The distributed implementation using four processes achieved the lowest execution time for graphs with 500 and 1000 vertices. At 2000 vertices, the performance of the parallel and distributed implementations was similar, with execution times of 5318.196 ms and 5111.105 ms, respectively.

For the largest tested graph with 3000 vertices, the parallel implementation achieved the lowest execution time of 8365.612 ms, compared with 9915.939 ms for the distributed implementation. This shows that distributed execution does not necessarily result in better performance. The distributed implementation introduces

additional communication and synchronization between processes, while the shared-memory parallel implementation avoids much of this inter-process communication.

Overall, the results demonstrate that both approaches provide significant performance improvements over sequential execution. Their relative performance depends on the size of the graph and the overhead introduced by task management, communication, and synchronization.

6 Conclusion

This project implemented and evaluated sequential, parallel, and distributed versions of the Fruchterman–Reingold force-directed graph layout algorithm. The implementations were compared using randomly generated graphs with varying numbers of vertices and edges to examine how different computational approaches affect execution time and scalability.

The results showed that the parallel implementation provided substantial performance improvements over the sequential version, particularly as the number of vertices increased. Increasing the number of edges had a smaller effect on execution time, which is consistent with the quadratic complexity of the repulsive-force calculation with respect to the number of vertices. The distributed implementation also reduced execution time when multiple processes were used, although its scaling was not linear. Communication and synchronization overhead limited the efficiency of additional processes, and in some configurations three processes performed more efficiently than four.

In the direct comparison of all three implementations, the distributed implementation with four processes achieved the lowest execution times for some of the smaller tested configurations, while the shared-memory parallel implementation achieved the lowest execution time for the largest tested graph.

Overall, the results demonstrate that parallel and distributed computation can improve the performance of force-directed graph layout, but the benefit depends on the size of the computational workload and the overhead introduced by the chosen approach.

7 Future Work

Future work could focus on improving the scalability of both the parallel and distributed implementations. For larger graphs, the quadratic cost of repulsive-force calculations could be reduced using approximation techniques instead of calculating interactions between every pair of vertices.

The distributed implementation could also be improved by reducing communication overhead and distributing additional parts of the computation more efficiently among processes. Further experiments on larger graphs and systems with more processing cores or multiple machines could provide a more complete evaluation of scalability.

Another possible extension would be to introduce a convergence-based stopping condition instead of using a fixed number of iterations. This would allow execution to stop automatically once the graph layout becomes sufficiently stable, rather than always performing a predefined number of iterations.